

ESP-NOWMIDIBridge ESP-NOW Client API

This document describes how to implement an ESP-NOW client to send MIDI events to the ESP-NOWMIDIBridge.

Overview

The bridge listens for ESP-NOW frames and translates them to MIDI events according to a configurable mapping table. All ESP-NOW frames follow a common envelope format:

[MAGIC (4 bytes)] [TYPE (1 byte)] [PAYLOAD (0-245 bytes)]

- > MAGIC: Always 0x4D 0x4A 0x41 0x4D (ASCII "MJAM")
- > TYPE: Message type (see below)
- > PAYLOAD: Type-specific data

Message Types

Type	Value	Direction	Description
JOIN	0x01	Client → Bridge	Request to join the bridge
LEAVE	0x02	Client → Bridge	Notify disconnection
DATA	0x03	Client → Bridge	MIDI event data
ADVERTISEMENT	0x04	Bridge → Client	Bridge availability broadcast
JOINED_ACK	0x05	Bridge → Client	Acknowledge successful join
INSTRUMENTS	0x06	Client → Bridge	Instrument list (future)

Type	Value	Direction	Description
NOTE_KEEPALIVE	0x07	Client → Bridge	Keepalive for held notes

Client Connection Sequence

1. Listen for Advertisement

The bridge broadcasts ADVERTISEMENT messages periodically (default: every 2500 ms) on channel 1.

Payload: [ROOM_ID] (1 byte)

- Only join bridges matching your configured ROOM_ID (default: 0x01)

2. Send JOIN

Send a JOIN message to the bridge MAC address.

Payload: [ROOM_ID] (1 byte)

The bridge will respond with JOINED_ACK containing the same ROOM_ID.

3. Send DATA Frames

Clients send DATA frames to the bridge MAC address. The payload is interpreted according to the bridge's mapping configuration.

Payload Format (configurable per mapping rule):

Typical MIDI mapping expects:

- Byte 0: Protocol identifier or command byte
- Byte 1+: Event-specific data (note, velocity, CC value, etc.)

The bridge's MappingRules decode the payload based on:

- Matching byte patterns (offset, mask, expected value)

- > Optional sender MAC filtering
- > Target MIDI channel and event type
- > Data byte derivation (from payload, sender MAC, or literals)

Example payloads:

- > [0x10, note_value, velocity] /rightarrow Note On on MIDI channel 0
- > [0x20, note_value, 0] /rightarrow Note Off
- > [0x30, cc_number, cc_value] /rightarrow Control Change

4. Note Keepalive

When a NOTE_ON is sent, the bridge starts tracking the note. The client must send periodic NOTE_KEEPALIVE frames to indicate the note is still held.

Client Sends Keepalive Payload: [channel, note] (2 bytes)

- > channel: MIDI channel (0–15)
- > note: Note number (0–127)

Keepalive Tolerance: The bridge tolerates occasional missed keepalives due to WiFi packet loss. It only sends a NOTE_OFF after missing N consecutive keepalives (default: 3). Each time a keepalive is received, the miss counter resets to zero.

Example: With a 100ms timeout and 3-miss threshold:

- > Client sends keepalive at t=0, 50, 100, 150, ...
- > If keepalive at t=100 is lost, miss counter becomes 1 (bridge sees t=150)
- > If keepalive at t=150 is lost, miss counter becomes 2
- > If keepalive at t=200 is lost, miss counter reaches 3 /rightarrow bridge sends NOTE_OFF
- > If keepalive at t=200 arrives, miss counter resets to 0 (note continues)

This prevents notes from being stuck if:

- > The client crashes or loses power
- > The wireless connection is lost for 300ms+ (3 misses × 100ms)

- > The user forgets to send NOTE_OFF

To keep a note playing, the client must send keepalive packets more frequently than the timeout interval. With default settings (100ms timeout, 3-miss threshold), notes tolerate up to ~300ms of connection loss.

5. Send LEAVE (Optional)

When disconnecting, send a LEAVE message.

Payload: (empty)

Peers are automatically removed after the peer timeout (default: 30000 ms) if inactive.

Frame Constants

```
// Protocol
MAGIC_BYTES           = "MJAM" (0x4D 0x4A 0x41 0x4D)
ROOM_ID               = 0x01 // Peer discovery identifier
MAX_PAYLOAD_SIZE      = 250 // Max ESP-NOW payload
MAX_SYSEX_SIZE        = 246 // Max SysEx data

// Timing
ADVERTISEMENT_INTERVAL_MS = 2500 // ms
PEER_TIMEOUT_MS           = 30000 // ms
STUCK_NOTE_TIMEOUT_MS     = 60000 // ms (configurable)
NOTE_KEEPALIVE_TIMEOUT_MS = 100
// ms (configurable, 0 = disabled)
KEEPALIVE_MISS_THRESHOLD  = 3
// consecutive misses before NOTE_OFF (configurable)
```

All values are configurable via `Kconfig.projbuild` in the firmware.

Implementation Examples

```
def join_bridge():  
    # Wait for ADVERTISEMENT frames on channel 1  
    while not received_advertisement:  
        listen_for_esp_now_frames()  
  
    # Extract bridge MAC from advertisement frame  
    bridge_mac = advertisement.sender_mac  
  
    # Send JOIN  
    frame = encode_frame(type=JOIN, payload=[ROOM_ID])  
    esp_now_send(bridge_mac, frame)  
  
    # Wait for JOINED_ACK  
    while not received_joined_ack:  
        listen_for_esp_now_frames()
```

```
def send_note_on(channel, note, velocity):  
    # Payload depends on bridge mapping configuration  
    # Example: [0x10, channel << 4 | note, velocity]  
    payload = [0x10, channel << 4 | note, velocity]  
    frame = encode_frame(type=DATA, payload=payload)  
    esp_now_send(bridge_mac, frame)
```

```
def send_keepalive(channel, note):  
    # Send periodic keepalive to keep note alive  
    payload = [channel, note]  
    frame = encode_frame(type=NOTE_KEEPALIVE, payload=payload)  
    esp_now_send(bridge_mac, frame)
```

```
def send_note_off(channel, note):
    payload = [0x20, channel << 4 | note, 0]
    frame = encode_frame(type=DATA, payload=payload)
    esp_now_send(bridge_mac, frame)
```

```
def client_note_hold_loop():
    # While holding a note, send keepalives at regular
    # intervals
    send_note_on(0, 60, 100) # Start the note

    keepalive_interval = 50 # ms, should be less than
    # bridge's timeout
    last_keepalive = current_time_ms()

    while note_is_held_by_user:
        if current_time_ms() - last_keepalive >= keepalive_interval:
            send_keepalive(0, 60)
            last_keepalive = current_time_ms()

        # Check for user release
        if not note_is_held_by_user:
            break

    send_note_off(0, 60) # Release the note
```

Mapping Configuration

The bridge's MappingRules array defines how incoming ESP-NOW payloads translate to MIDI events. Each rule specifies:

- > Match Criteria: Payload byte(s) to match (offset, mask, expected value)
- > Optional Sender MAC Filter: Only apply rule if sender matches (or match any sender if nil)
- > Target MIDI: Channel and event type
- > Data Byte Sources: Where to get MIDI data1/data2 from:
 - > Fixed literal value

- > Payload byte at offset
 - > Payload nibble (high/low)
 - > Sender MAC byte at offset
 - > Unused
- > SysEx Data: Optional range of payload bytes to forward as SysEx data

Rules are evaluated in order; the first matching rule is applied.

Notes for Client Implementers

1. Bridge Discovery: Scan for ADVERTISEMENT frames to find the bridge MAC address
2. Peer Management: The bridge removes peers after 30 seconds of inactivity
3. Note Keepalive: Send keepalive packets more frequently than the keepalive timeout (default: 100ms). Example: send keepalive every 50–80ms while a note is held
4. Keepalive Tolerance: The bridge tolerates N consecutive missed keepalives before releasing a note (default: 3). This means a connection can be down for roughly $N \times \text{timeout_ms}$ before a note is released
5. Stuck Notes: Send NOTE_OFF promptly; if you forget or the connection drops:
 - > The bridge will send NOTE_OFF after N consecutive missed keepalives (if keepalive monitoring is enabled)
 - > OR after the stuck-note timeout (default: 20 seconds) if keepalive is disabled
6. Multiple Clients: Clients must have unique source MACs; the bridge supports up to 10 concurrent peers (configurable)
7. WiFi Coexistence: ESP-NOW shares the WiFi channel; ensure bridge and clients use the same channel
8. Keepalive Timing: Keep keepalive interval significantly below the timeout to account for WiFi latency and jitter. A safe margin is 2–3× the timeout. Example: if timeout is 100ms, send keepalives every 30–50ms

References

- > Bridge Firmware: [ESPNOw/InstrumentBridge](#)
- > Mapping Configuration: Edit `main/Translator.swift` in the firmware
- > Configuration: Edit `main/Kconfig.projbuild` or use `idf.py menuconfig`